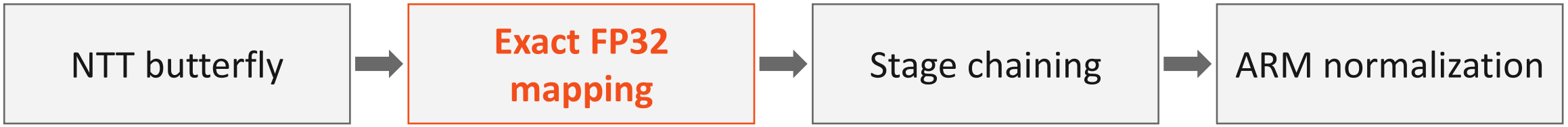


Implementation and Evaluation of

NTT Arithmetic for ML-KEM on a CGLA

Takuto Ando and Yasuhiko Nakashima

Nara Institute of Science and Technology (NAIST)



NTT Arithmetic on an Existing Datapath

Exact modular arithmetic without an NTT-specific unit

- Polynomial multiplication relies on repeated butterflies.

$$x = (a + wb) \bmod q$$

- Coefficient pairing and twiddle assignment change at every stage.

$$y = (a - wb) \bmod q$$

- The mapped arithmetic must preserve exact residues.

N = 256 q = 3329

Reuse the same programmable array used for non-cryptographic kernels.

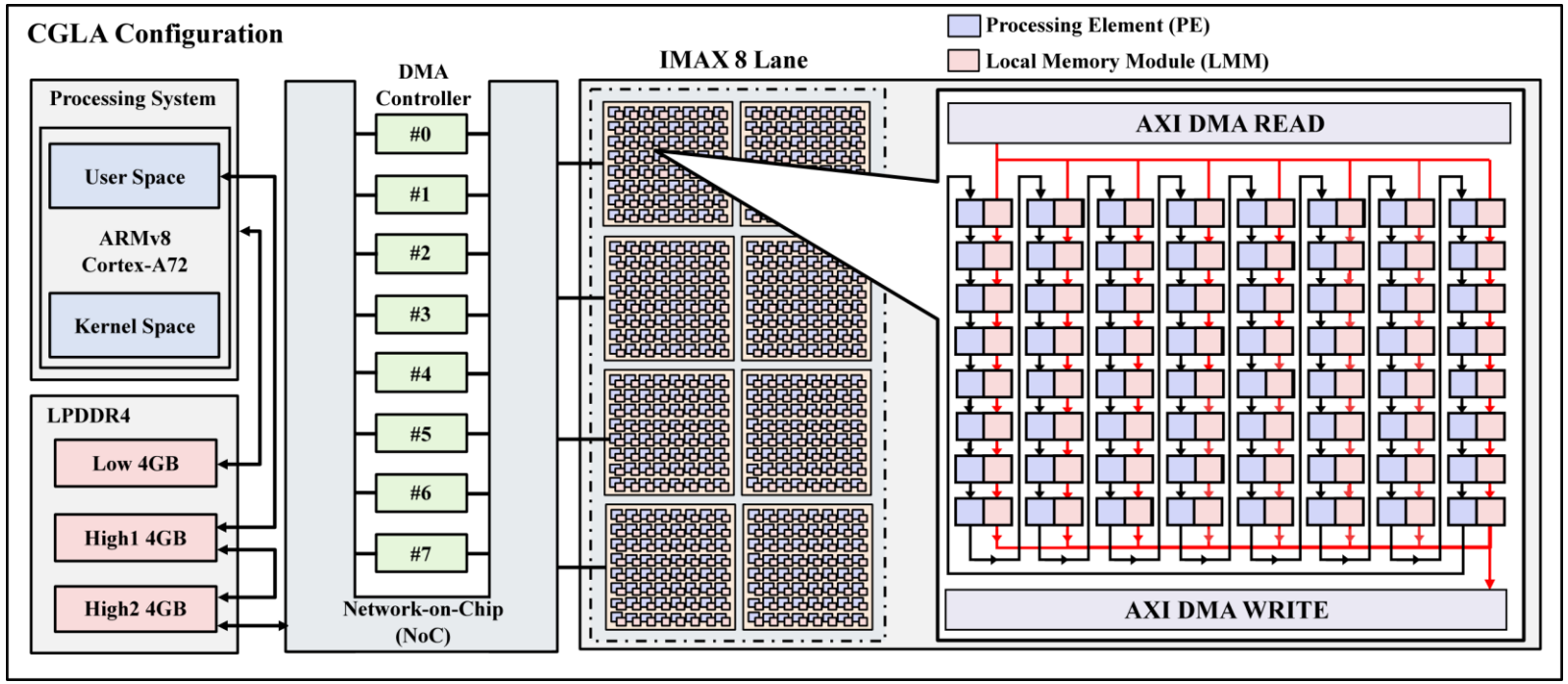
Transform Scope

	Evaluated CGLA driver	**FIPS 203 ML-KEM**
Modulus	$q = 3329$	$q = 3329$
Transform	8-stage cyclic radix-2	7-layer incomplete negacyclic
Polynomial ring	$x^{256} - 1$	$x^{256} + 1$
Scope	Forward driver and stage records	NTT, pairwise multiplication, INTT

This work evaluates arithmetic over the ML-KEM modulus.

The current CGLA path is not the standardized complete ML-KEM multiplication path.

IMAX Configuration



Host / accelerator

- ARM controls DMA and kernel launch.
- PEs and LMMs form a linear pipeline.
- The compiler assigns operations to ordered rows.

CPU-Grounded Linear Array (CGLA) · a programmable ASIC architecture

FPGA prototype · AMD Versal VPK180 · 145 MHz · 64-stage pipeline

Mapping Requirements



Interface	Record / responsibility
Coefficient	Two FP32 bit patterns packed as [a | b] in 64 bits
Twiddle	Paired [w_lo | w_hi] record
CGLA output	Corrected residues at next-stage addresses
ARM boundary	Normalize integer-near FP32 records

Arithmetic and stage layout must both fit the existing execution interface.

Split the Twiddle Before Reduction

$$w = w_lo + 256 w_hi$$

Low part
$p_lo = b \times w_lo$

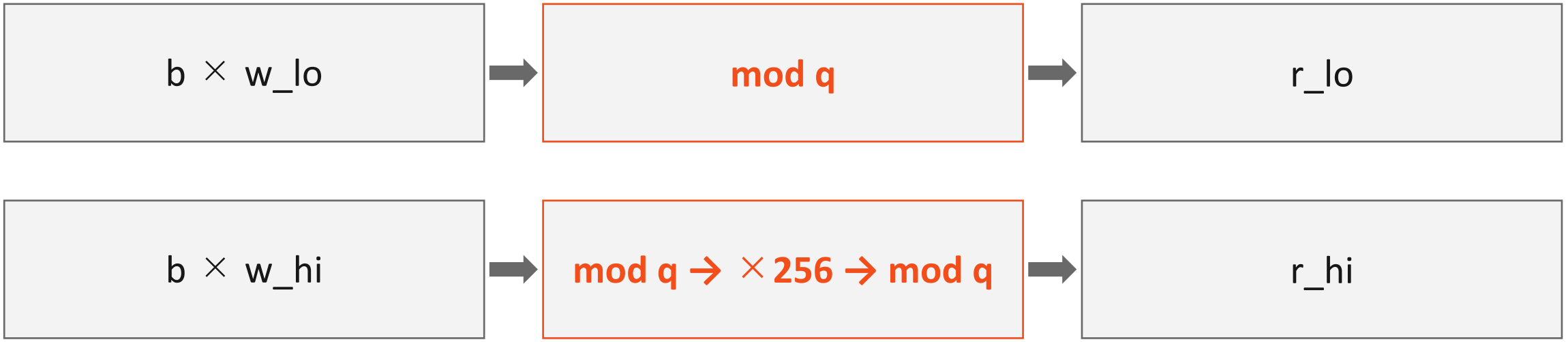
High part
$p_hi = b \times w_hi$

$w_lo \in [0, 255]$
$p_lo \leq 848{,}640$

$w_hi \in [0, 13]$
$p_hi \leq 43{,}264$

Both integer products remain below 2^{24} and are exactly representable.

Reduce Before Recombination



$$t = (r_lo + r_hi) \bmod q, \quad r_lo + r_hi < 2q$$

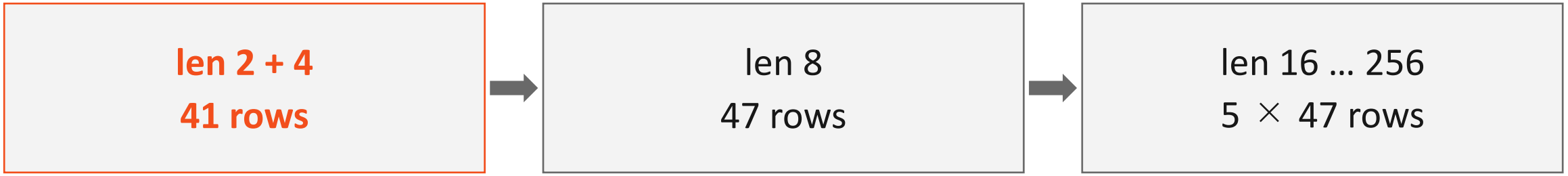
Direct partial-product addition produced one-ULP errors on the tested FPGA path.

Separate reductions avoid the observed direct-recombination failure.

This is a prototype observation, not a limitation of correctly rounded IEEE FP32 arithmetic.

Seven Calls for Eight Stages

Fuse the fixed first two stages; reuse one generic schedule afterward



Fused call

115 instructions, rows 0–40

Fixed twiddles {1, 1, 1, 1729}

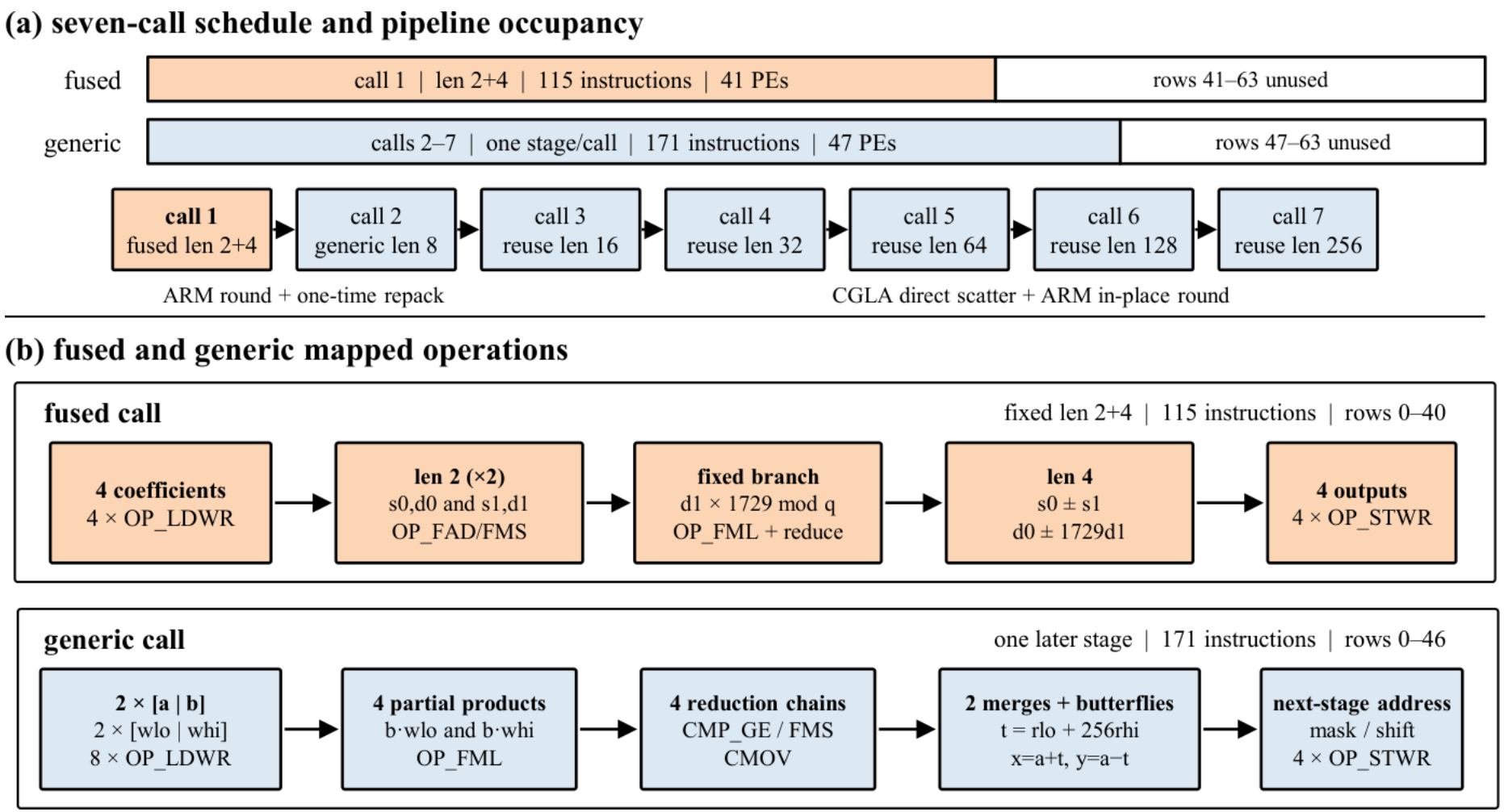
Generic call

171 instructions, rows 0–46

Two butterflies per iteration

Five calls after len 8 reuse the loaded generic configuration.

Mapped Butterfly Operations



Four independent residue chains fill dependency gaps in the generic call.

Record Layout Across Stage Boundaries

After the fused call



Five subsequent generic transitions



Direct scatter removes rearrangement; host normalization remains.

ARM uses vrndn_f32 on both lanes; it does not repeat modular multiplication or reduction.

Experimental Setup

Item	Setting
Platform	VPK180 FPGA, 145 MHz, 64-stage CGLA
Host reference	ARM Cortex-A72; pure-C Cooley–Tukey + Barrett reduction
Workload	N = 256, q = 3329; batch 8 and batch 64
Timing	ARM preparation + CGLA phases + output handling
Statistics	21 retained runs per result, including slow trials
Reporting	Time per NTT is batch normalized

Every hardware run is checked coefficient by coefficient.

Correctness Checks

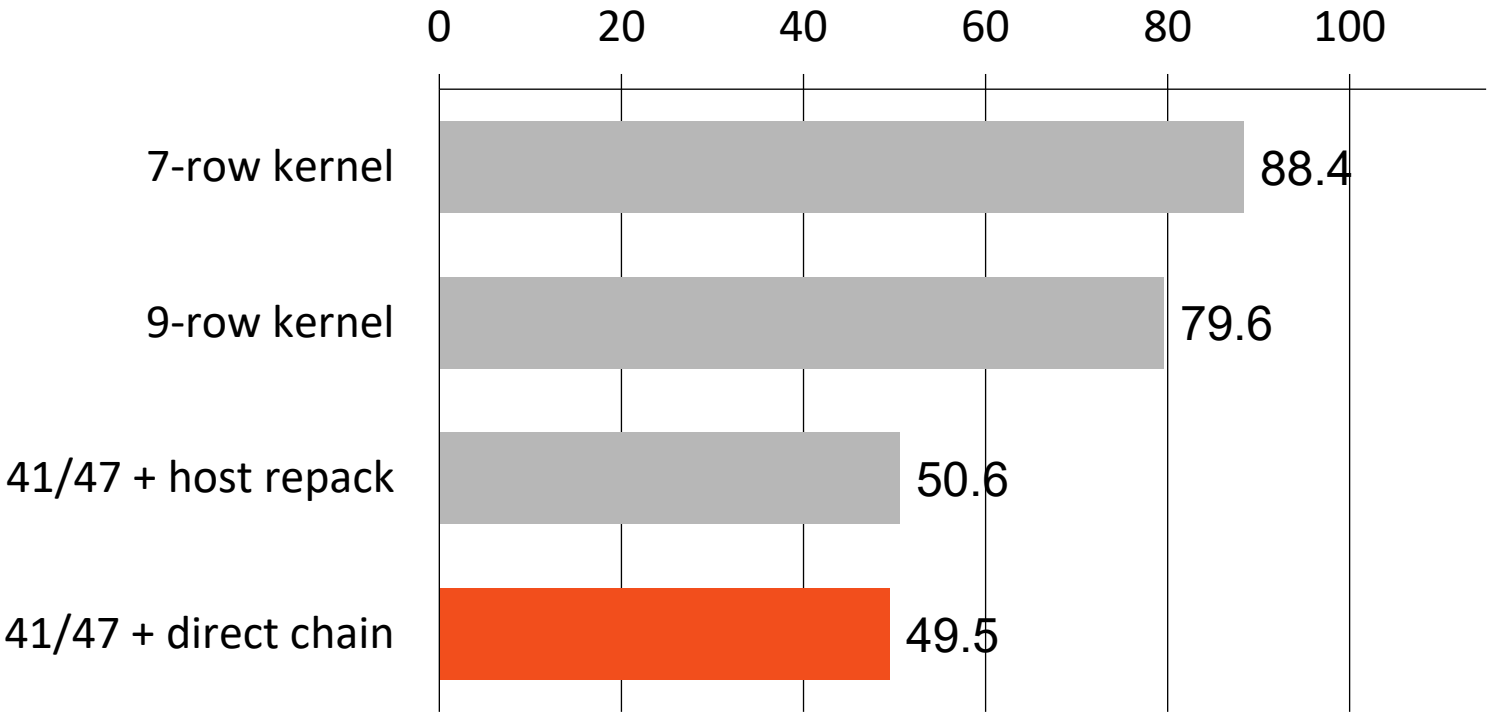
Check	Coverage	Result
FPGA, four cohorts	105 runs / 817,152 coefficients	All match CPU reference
Host split-product oracle	11,082,241 cases	Exact residue construction
Host butterfly correction	11,082,241 (a, t) pairs	Exact bounded correction

Host arithmetic coverage and FPGA input cohorts test different boundaries.

The retained FPGA runs validate the mapped stage chain on tested inputs.

How Much Work Should Be Offloaded?

Measured FPGA time (µs/NTT), batch 8



7 → 9 rows
Output packing

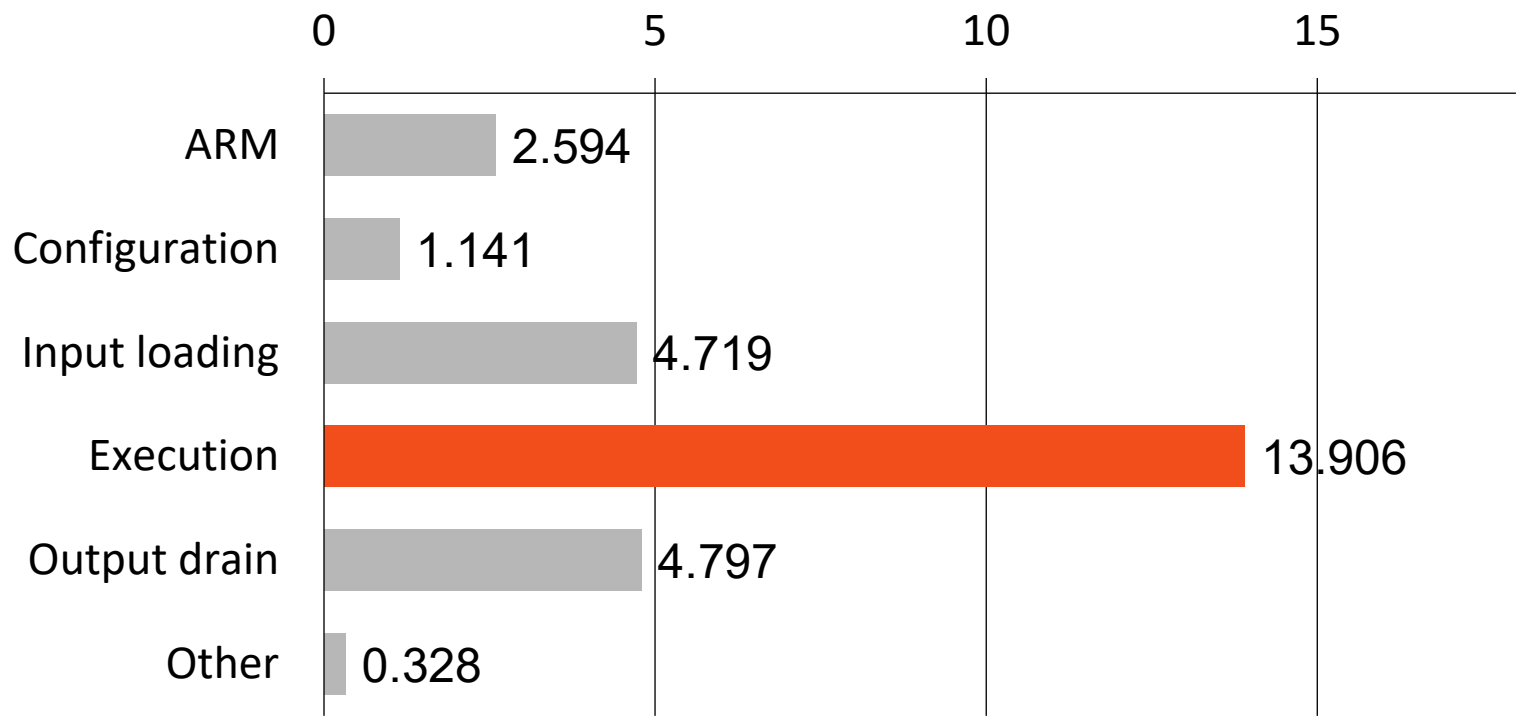
Host repack → direct chain
Lower latency in
17 of 21 pairs

The larger change includes arithmetic offload, fusion, and stage chaining.

21 runs per implementation; same FPGA E2E protocol. This is not a PE-count-only comparison.

Where Does Batch-64 Time Go?

FPGA phase medians (µs/NTT)



27.5 µs / NTT
Measured E2E

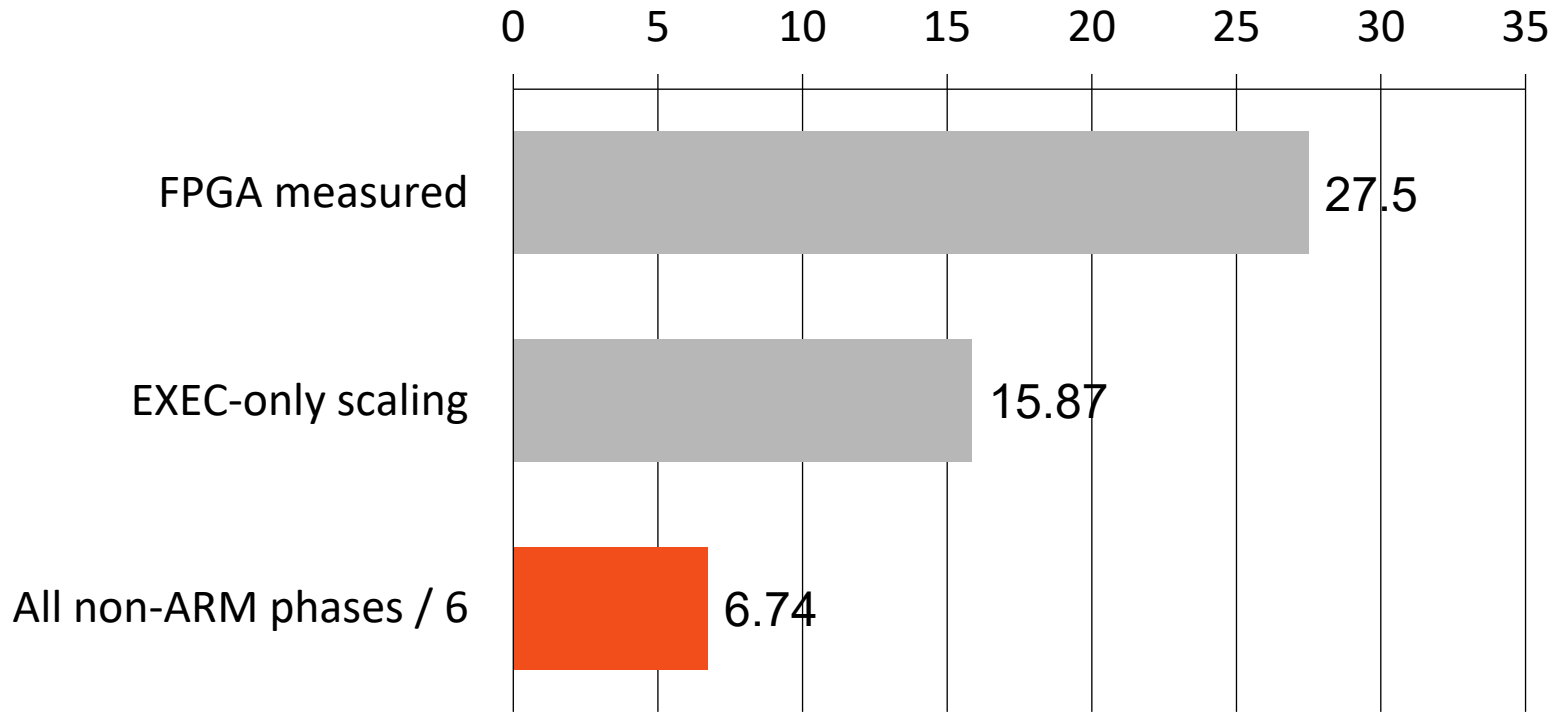
Batch 64
1.76 ms per group
21-run median

Configuration, transfers, and host work remain outside arithmetic execution.

Phase medians need not sum to the median total.

ASIC Timing Depends on Interface Scaling

Batch-normalized time (µs/NTT)



EXEC-only

Keep configuration and transfer intervals unchanged.

All-phase scenario

T_ARM + T_CGLA / 6

6.74 µs is the optimistic all-phase projection, not a silicon measurement.

Batch Size and Local Memory

	Batch 8	Batch 64
Live set	17 KB	129 KB
LMM synthesis point	32 KB	256 KB
Generic active-power estimate	1.35 W	23.4 W
Projected system energy	10.1 µJ/NTT	58.9 µJ/NTT

Batch 8 minimizes modeled energy; batch 64 minimizes measured amortized time.

The energy ordering depends on the stated LMM synthesis estimates.

PE-gated ASIC + phase-accounted ARM model. No direct system-power measurement.

Summary

Exact arithmetic and explicit stage transitions on a programmable array

- Implemented split-residue arithmetic and a seven-call 41/47-row schedule.
- Matched 817,152 coefficients across 105 FPGA runs.
- Measured 27.5 µs/NTT at batch 64; ASIC timing depends on interface scaling.
- Next, adapt the schedule to the seven-layer FIPS 203 NTT/INTT path.



Thanks!

Do you have any questions?

Takuto Ando · Nara Institute of Science and Technology

Backup · GPU Context and Scope

	CGLA	Archived RTX 4090
Transform	8-stage cyclic driver	7-layer CUDA benchmark
Batch context	64	4096
Amortized time	6.74 µs projected	0.13 µs archived
Evidence	Coefficient checks per run	No same-run identity/build hash; checks skipped

These rows provide historical context, not a matched-transform comparison.